CS433: Parallel Programming Assignment-1 Report (Group 8)

Aviral Gupta (230246) **Aruz Awasthi (230209)**

Yatharth Sharma (231198)

1 Minimum Vertex and Edge Coloring – OpenMP

1.1 Problem Statement

The objective is to find the minimum number of colors needed to color both the vertices and edges of an undirected graph $G = (V, E)$. Let $n = |V|$ and $m = |E|$. A valid assignment must satisfy:

1. Adjacent vertices receive different colors.
2. Every edge receives a color different from the colors of both of its endpoint vertices.
3. The three edges of every triangle receive distinct colors.

Edges sharing a vertex are not required to have different colors unless they belong to a triangle together. Our implementation enforces exactly the constraints specified above.

The program uses OpenMP and accepts the input filename, output filename, and number of threads in that order. It returns one valid minimum-color assignment, using integer color identifiers beginning at zero.

1.2 Previous Attempt: Parallelizing Candidate Color Counts

Our initial approach distributed different candidate numbers of colors among threads. Each thread took a candidate value k dynamically and independently checked whether a valid assignment using at most k colors existed. After the feasibility checks, we selected the smallest candidate value with a successful coloring.

This approach parallelized the outer loop over candidate color counts, but each individual coloring search remained sequential. In our initial experiments, speedup was mostly close to one and was often below one. Parallelization overhead and unequal search costs across candidate values can explain this behavior. In particular, an unsuccessful feasibility check may require exploring a large search tree, while an easier candidate may finish quickly. Assigning an entire difficult candidate to one thread leaves that search as a potential bottleneck. Furthermore, finding a feasible larger candidate does not establish optimality until all smaller candidate values have been ruled out.

We therefore changed the parallelization strategy to distribute work within the coloring search for a fixed k, allowing multiple threads to cooperate on the same feasibility check.

1.3 Final Implementation

1.3.1 Conflict Graph Representation

We convert the problem into an ordinary vertex-coloring problem on a conflict graph H. Each original vertex and each original edge becomes one object in H, giving $M = n + m$ objects in total. Original vertices use object identifiers $0, \ldots, n - 1$, and original edge e uses object identifier $n + e$.

Two objects are connected in H exactly when their colors must differ. We construct the following conflicts:

Preprint.

- **Vertex–vertex conflicts:** For every original edge (u, v), connect the objects corresponding to u and v.
- **Vertex–edge conflicts:** Connect each original edge object to both of its endpoint vertex objects.
- **Triangle edge conflicts:** For every triangle, connect each pair among its three edge objects.

To find triangle conflicts for an edge (u, v), we intersect the sorted adjacency lists of u and v using two pointers. Every common neighbor w identifies a triangle (u, v, w). The edge object for (u, v) then receives conflicts with the edge objects for (u, w) and (v, w). Processing all edge objects establishes all required triangle conflicts.

This transformation preserves feasibility: an assignment satisfies the original constraints if and only if it is a proper coloring of H.

1.3.2 Sequential Algorithm: Exact Backtracking

We test candidate values $k = 1, 2, \ldots, M$ in increasing order. For each candidate, we search for a proper coloring of H using at most k colors. The first successful candidate is the minimum.

We use a fixed object order. For each original vertex u, we place its vertex object into the order, followed by the edge objects joining u to lower-numbered vertices. Every original edge appears exactly once, after both endpoint vertices have been placed in the order.

The backtracking search proceeds as follows:

1. Initialize every object as uncolored, represented by -1.
2. Select the next object in the fixed order and try an eligible color. Reject a color if an already colored conflict neighbor uses it.
3. After a valid assignment, advance to the next object.
4. If no eligible color remains, backtrack to the previous object and try its next eligible color.
5. A complete assignment proves feasibility. Exhausting all branches proves that the candidate k is insufficient.

The implementation uses iterative backtracking rather than recursive function calls. Arrays store the current color assignment and the number of color labels introduced at each search depth.

1.3.3 Color Symmetry Reduction

Permuting the color labels does not change whether an assignment is valid. Exploring every such permutation would repeat equivalent work. We therefore introduce color labels in order.

If q labels have already been used, the next object may use an existing label $0, \ldots, q-1$, or introduce the next label q if $q < k$. It cannot introduce a higher-numbered unused label. The first object is consequently assigned color zero.

This restriction preserves a representative of every feasible coloring: any valid assignment can be relabeled according to the order in which its colors first appear. It reduces redundant branches without changing the minimum number of colors.

1.3.4 Parallel Algorithm: Search Tree Decomposition

For a fixed candidate k, we divide the search tree into independent subtrees using partial color assignments, called prefixes.

1. **Prefix generation:** Starting with the empty prefix, sequentially expand all valid prefixes by one search level at a time. Apply the same conflict checks and color symmetry restriction used in the full search.
2. **Adaptive split depth:** Continue expanding until the number of prefixes reaches a target of $4p$, where p is the requested thread count, or until all objects are assigned. Expansion also stops if no valid prefixes remain. The target is therefore not guaranteed to be reached.
3. **Parallel subtree search:** Enter an OpenMP parallel region and distribute the prefixes with `schedule(dynamic, 1)`. Each prefix defines the fixed initial assignments for an otherwise independent backtracking search.
4. **Result handling:** If a thread finds a full coloring, it saves the assignment and sets a shared atomic success flag. Other threads skip new tasks after observing the flag and periodically check it while searching their current subtrees.
5. **Next candidate:** If every subtree fails, increase k and repeat. Otherwise, return the successful assignment and the current candidate value.

Dynamic scheduling allows a thread that finishes a small subtree to take another prefix while other threads continue searching larger subtrees. It is useful because different valid prefixes can lead to substantially different amounts of backtracking. A single long subtree can still limit load balance because tasks are not split further during their search.

Candidate color counts remain sequential in the final implementation; the parallel work is inside each feasibility test. Unlike the persistent parallel region used in Q2, this implementation enters a new parallel search region for each candidate whose prefix list is nonempty. This adds parallel-region overhead, particularly when individual feasibility checks are short.

1.3.5 Parallel Conflict Graph Construction

Conflict graph construction is also parallelized using `omp parallel for` with `schedule(dynamic, 64)`. Each iteration constructs one object's conflict list. The original graph is read-only during this phase, and each conflict list has a single writer, so no critical section is required for construction.

1.3.6 Performance Optimizations

- **Parallelism within a candidate:** Multiple threads cooperate on the same feasibility check, including expensive checks that must prove a candidate impossible.
- **Adaptive prefix decomposition:** The split depth is chosen according to the available valid prefixes and thread count, rather than using a fixed depth for every graph.
- **Dynamic scheduling:** Prefixes are assigned one at a time, allowing threads to pick up additional work as they finish.
- **Color symmetry reduction:** Canonical introduction of labels avoids searching assignments that differ only by renaming colors.
- **Early conflict rejection:** A candidate color is rejected as soon as a conflicting neighbor using that color is found.
- **Thread-local reusable state:** Each thread reuses its color and depth-state arrays across subtree tasks and candidate values.
- **Cooperative early termination:** Once a valid coloring is found, the atomic flag prevents unnecessary new searches and eventually stops searches already in progress.
- **Sorted adjacency lists:** Two-pointer intersections identify common neighbors without testing every possible third vertex.

1.4 Performance Evaluation

1.4.1 Benchmark Method

We consider six graph instances with vertex and edge counts $(N, m) = (4, 6)$, $(10, 39)$, $(12, 45)$, $(8, 28)$, $(15, 45)$, and $(20, 73)$. Here, N denotes the number of vertices and m denotes the number of edges.

Each available graph and thread-count combination was run five times with $p = 1, 2, 4, 8, 16, 24, 32$ threads. The tables report average execution time in seconds and speedup relative to the same solver using one thread:

$$\overline{T_G(p)} = \frac{1}{5}\sum_{i=1}^{5} T_{G,i}(p), \qquad S_G(p) = \frac{\overline{T_G(1)}}{\overline{T_G(p)}}.$$

The reported solver time includes conflict-graph construction, prefix generation, and the searches over candidate color counts. Input reading, initial object ordering, initial allocation of thread-local search arrays, and output writing are outside the timed region. Temporary allocations during conflict construction and prefix generation are included.

1.4.2 Results for N = 4, m = 6

Threads	Average time (s)	Speedup
1	0.0000282920	1.000
2	0.0000734346	0.385
4	0.0001446900	0.196
8	0.0002931330	0.097
16	0.0006662640	0.042
24	0.0010436700	0.027
32	0.0191172000	0.001

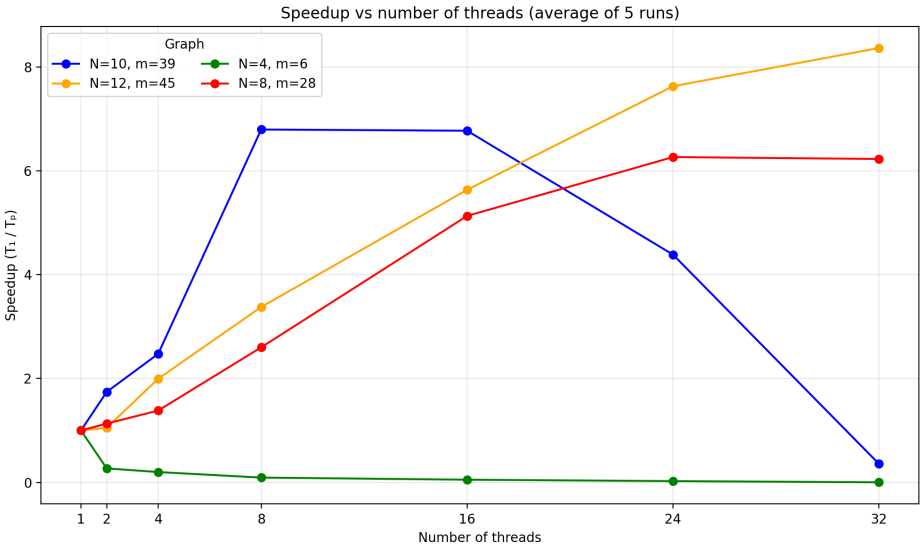


Figure 1: Speedup versus number of threads for the graph instances.

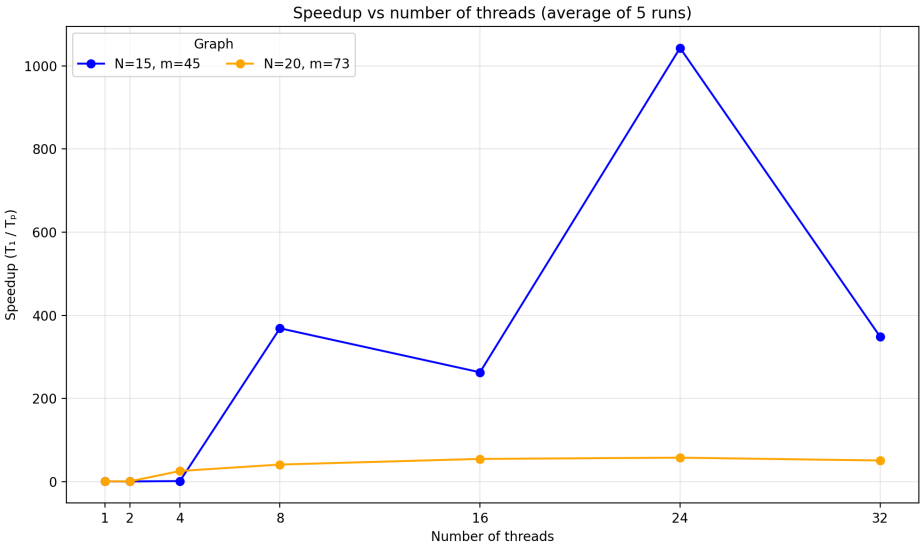


Figure 2: Superlinear speedup observed in the parallel coloring search.

1.4.3 Results for N = 8, m = 28

Threads	Average time (s)	Speedup
1	5.671790	1.000
2	5.174980	1.096
4	4.122080	1.376
8	2.224790	2.549
16	1.057040	5.366
24	0.810911	6.994
32	0.796189	7.124

1.4.4 Results for N = 12, m = 45

Threads	Average time (s)	Speedup
1	4.652630	1.000
2	4.228710	1.100
4	2.661240	1.748
8	1.283700	3.624
16	0.838928	5.546
24	0.604573	7.696
32	0.615961	7.553

1.4.5 Results for N = 15, m = 45

Threads	Average time (s)	Speedup
1	4.41611000	1.000
2	6.28093000	0.703
4	2.74931000	1.606
8	0.01197150	368.884
16	0.01678380	263.117
24	0.00423457	1042.870
32	0.01266420	348.707

1.4.6 Results for N = 20, m = 73

Threads	Average time (s)	Speedup
1	6.538250	1.000
2	6.676340	0.979
4	0.253078	25.835
8	0.158205	41.328
16	0.119171	54.865
24	0.112984	57.869
32	0.128115	51.034

1.4.7 Discussion of Observed Trends

- **Parallel overhead on a small graph:** For $N = 4, m = 6$, execution time increases from approximately $28.3\,\mu$s with one thread to 19.1 ms with 32 threads. The search is too short to benefit from additional threads. Parallel-region entry, scheduling, and synchronization can outweigh the useful parallel work. The particularly large increase at 32 threads may also involve system contention or timing variability, which these averages alone cannot distinguish.
- **Benefit from substantial search work:** For $N = 8, m = 28$, runtime decreases from 5.671790 s to 0.796189 s, giving a speedup of 7.124 at 32 threads. For $N = 12, m = 45$, the best measured time is 0.604573 s at 24 threads, corresponding to a speedup of 7.696. These graphs provide enough backtracking work for parallel subtree exploration to improve performance.
- **Superlinear speedup for N = 15, m = 45:** This graph exhibits superlinear speedup, meaning that the speedup exceeds the number of threads. At eight threads, the measured speedup is 368.884. At 24 threads, runtime decreases from 4.416110 s to 0.00423457 s, giving a speedup of 1042.870.

  A backtracking search need not explore the same number of assignments at every thread count. Parallel exploration can reach a successful branch earlier and use the shared success flag to cancel other searches. A sequential traversal may

explore many unsuccessful branches before reaching that solution. Furthermore, the prefix target is $4p$, so changing the thread count can change the split depth and search decomposition. These mechanisms provide a plausible explanation for the superlinear result. They do not imply that an identical fixed workload is executed over a thousand times faster. Search-node counts and per-candidate timings would be needed to confirm how much work was avoided.

- **Superlinear behavior for N = 20, m = 73:** This graph also shows superlinear speedup. At four threads, runtime decreases from 6.538250 s to 0.253078 s, giving a speedup of 25.835. The best measured configuration uses 24 threads, with a speedup of 57.869. Earlier discovery of a valid assignment and cancellation of remaining searches are plausible contributors here as well.
- **Nonmonotonic scaling:** More threads do not always reduce execution time. For $N = 15, m = 45$, two threads are slower than one, and 16 threads are slower than eight. Both this graph and $N = 20, m = 73$ are fastest at 24 threads and become slower at 32 threads. Uneven subtree sizes, changes in search order, sequential prefix generation, and scheduling overhead can produce this behavior. Hardware contention may also contribute.
- **Dependence on graph structure:** Vertex and edge counts alone do not determine search difficulty. The conflict structure, object order, and location of successful assignments influence how much backtracking is required. Consequently, a graph with more vertices or edges need not take longer to solve or show greater speedup.

2 System of Equations with Hourglass Matrix – OpenMP

2.1 Problem Statement

The objective is to develop a shared-memory parallel program to solve $Ax = y$, where A is a $(2n+1) \times (2n+1)$ real matrix and x and y are real vectors of length $2n+1$. We use OpenMP to compute the unknown vector x from the given matrix A and right-hand-side vector y, without explicitly inverting A.

Rows and columns are numbered from 0 to $2n$. The permitted non-zero entries in row i lie between columns i and $2n-i$, inclusive. Equivalently, defining

$$L_i = \min(i, 2n-i), \qquad R_i = \max(i, 2n-i),$$

we have $A_{i,j} = 0$ for $j < L_i$ or $j > R_i$. This gives the matrix its hourglass structure. We denote the matrix dimension by $N = 2n+1$, distinguishing it from the input parameter n.

2.2 Previous Attempts

We explored two approaches before adopting blocked elimination.

2.2.1 Attempt 1: Step-by-Step Outward Substitution

- **Sequential algorithm:** We first solved for the central variable $x[n]$. We then substituted the known central value into rows $n-1$ and $n+1$ to solve for the next pair of variables, continuing outward one pair at a time.
- **Parallel algorithm:** We attempted to parallelize the substitutions into the two target rows at each outward step.
- **Limitation:** Restricting updates to the immediate pair of rows provides limited parallel work at each step. Dependencies between successive pairs also require synchronization. These factors limit thread utilization and can make synchronization overhead large relative to the useful work.

2.2.2 Attempt 2: Column-Wise Elimination

- **Sequential algorithm:** After computing $x[n]$, we immediately subtracted its contribution from all remaining rows. We then solved for $x[n-1]$ and $x[n+1]$, subtracted their contributions from all unsolved rows, and continued outward.
- **Parallel algorithm:** Updates to different rows were distributed across threads. Since each row had a single writer during an update phase, these updates did not require critical sections.
- **Limitation:** With row-wise matrix storage, traversing a column accesses separate row allocations. For large matrices, repeated column passes can revisit data after its cache lines have been evicted. This motivated batching several variable contributions together when updating each row.

2.3 Final Attempt

2.3.1 Matrix Storage and Solution Structure

We store only the permitted non-zero interval of each row in a `vector<double>`. The mathematical coefficient $A_{i,j}$ is stored at `A[i][j - L_i]`. In particular, the central row contains a single entry, so the central coefficient is stored at `A[n][0]`.

The central equation gives

$$x[n] = \frac{y[n]}{A_{n,n}}.$$

At outward step k, let $l = n-k$ and $r = n+k$. The only variables present in these two rows are $x[l], \ldots, x[r]$. After substituting the already solved interior variables, we obtain

$$a\,x[l] + b\,x[r] = \widehat{y}_l,$$
$$c\,x[l] + d\,x[r] = \widehat{y}_r,$$

where $a = A_{l,l}$, $b = A_{l,r}$, $c = A_{r,l}$, and $d = A_{r,r}$. Using $\Delta = ad - bc$, the solution is

$$x[l] = \frac{\widehat{y}_l d - b\widehat{y}_r}{\Delta}, \qquad x[r] = \frac{a\widehat{y}_r - c\widehat{y}_l}{\Delta}.$$

This method assumes $A_{n,n} \neq 0$ and $\Delta \neq 0$ for each pair. The current implementation does not explicitly detect singular systems. Very small denominators can also affect numerical accuracy.

2.3.2 Sequential Algorithm: Blocked Elimination

We group outward steps into blocks of size $B = 10$. Each full block therefore solves ten pairs of variables. The final block may be smaller. The algorithm proceeds as follows:

1. **Initialization:** Compute $x[n]$ and subtract its contribution from the right-hand-side entries of every other row.
2. **Block processing:** For a block beginning at step b_s, set $b_e = \min(n, b_s + B - 1)$. Solve the pairs in increasing order of k, from b_s to b_e. Contributions from earlier blocks have already been removed from the stored right-hand side. Before solving the pair at step k, subtract contributions from pairs solved earlier in the current block into local right-hand-side values. Then solve the resulting 2×2 system using the formulas above.
3. **Bulk row updates:** After solving the block, update all unsolved pairs of rows with $k > b_e$. For each pair, accumulate the contributions of the newly computed variables and subtract them from the corresponding stored right-hand-side entries.
4. **Continuation:** Repeat until all n outward steps have been completed.

Blocking allows several coefficients from a row to be consumed during one visit. This improves spatial locality and reduces repeated row visits compared with updating one variable pair at a time.

Algorithm 1 Sequential algorithm for Blocked Bowtie Solver

Input: Matrix dimension N, Matrix A, Vector y, Block size C
Output: Vector x such that $Ax = y$
$n \leftarrow (N-1)/2$
$x[n] \leftarrow y[n]/A[n][0]$
times $\leftarrow n/C$ if $n\%C == 0$ else $n/C + 1$
for $i \in 0$ to *times* -1 **do**
    $bs \leftarrow i \times C + 1$
    $be \leftarrow \min(n, (i+1) \times C)$
    for $k \in bs$ to be **do**
        $l \leftarrow n - k, r \leftarrow n + k$
        for $previous \in bs$ to $k - 1$ **do**
            Subtract contributions of already computed variables in the block from $y[l]$ *and* $y[r]$
        end for
        Solve the 2×2 *system using Cramer's rule to find* $x[l]$ *and* $x[r]$
    end for
    for $row \in 0$ to $n - be - 1$ **do**
        for $current \in bs$ to be **do**
            Subtract contributions of current block variables from upper rows of y
        end for
    end for
    for $row \in n + be + 1$ to $N - 1$ **do**
        for $current \in bs$ to be **do**
            Subtract contributions of current block variables from lower rows of y
        end for
    end for
end for

2.3.3 Parallel Algorithm: Blocked Elimination

The pair solves within a block depend on previously solved pairs, so they are executed sequentially. Updates to the remaining row pairs are independent and are distributed across threads using OpenMP.

1. **Thread team initialization:** After computing the central variable, enter one parallel region with `num_threads(num_threads)`. The same team is used for the initial updates and all subsequent blocks.
2. **Central-variable updates:** An `omp for schedule(static)` loop subtracts the central variable's contribution from every outer pair of rows.
3. **Sequential block resolution:** For each block, `omp single` lets one thread perform the local substitutions and solve all pairs in that block. The selected thread need not be the same for every block.

4. **Parallel bulk updates:** An `omp for schedule(dynamic, 32)` loop distributes the remaining row-pair updates. Each iteration updates one upper row and its corresponding lower row using all newly solved variables from the block.

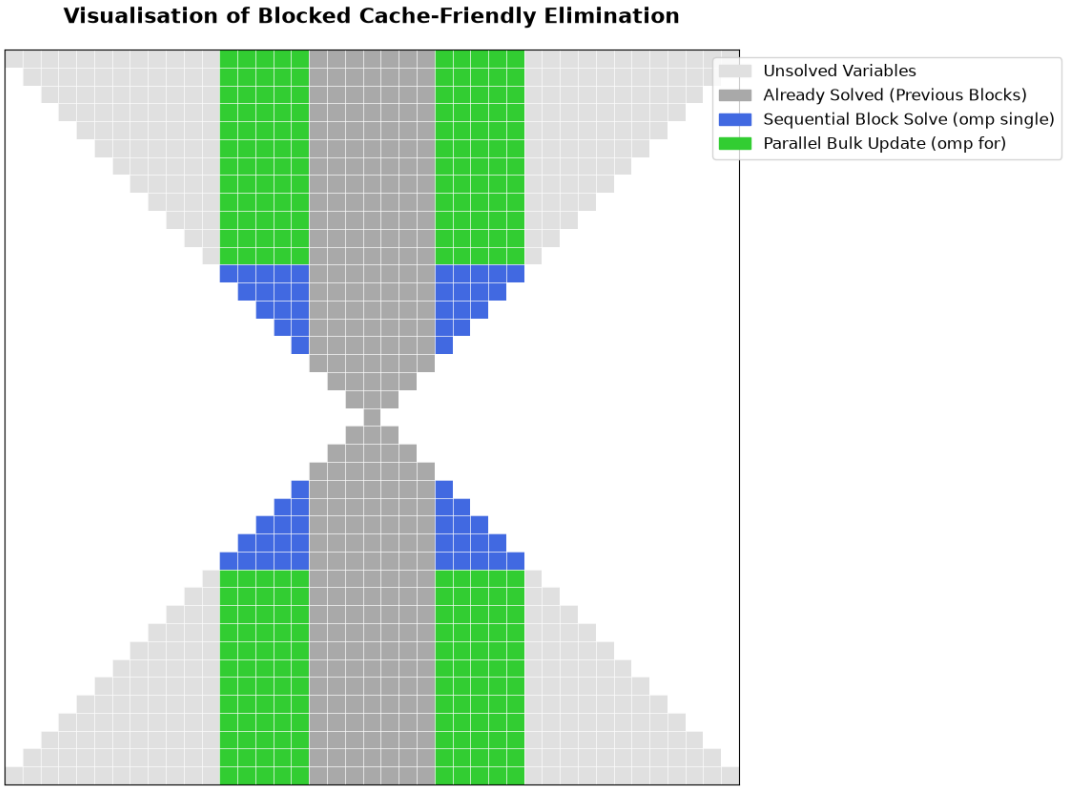


Figure 3: Blocked elimination with parallel updates to unsolved rows.

2.3.4 Performance Optimizations

- **Blocked processing:** Each row update handles several variable contributions together, improving locality and reducing the number of bulk-update phases.
- **Compressed row storage:** Entries outside the permitted interval are not stored, avoiding allocation and processing of structural zeros.
- **Cached row pointers:** The code obtains pointers such as `double *Al = A[l].data()` before inner loops. This simplifies repeated element access and may assist compiler optimization. Ordinary `vector::operator[]` does not perform bounds checking, so this is not a bounds-check elimination.
- **Persistent thread team:** One parallel region encloses all blocks, avoiding repeated parallel-region entry and exit.
- **Dynamic scheduling:** Bulk updates use `schedule(dynamic, 32)`, assigning work in chunks of 32 row pairs. Iterations within a block have similar arithmetic work, so any benefit over static scheduling must be established through measurement, accounting for scheduling overhead.
- **SIMD reduction:** The bulk-update inner loop uses an OpenMP SIMD directive with a reduction on `lu` and `ru` to expose the two contribution sums to compiler vectorization. Actual vectorization depends on the compiler and target hardware.

2.3.5 Block Size Selection

We selected the block size empirically by testing each integer block size C from 1 through 32 on an input of size $n = 2^{12} = 4096$. We compared execution times and observed the best performance for C between 10 and 13. Therefore, we selected $C = 10$ for the final implementation.

Smaller blocks require more update phases and synchronization, while larger blocks increase sequential work within each block. The selected size balances these costs in our benchmark environment.

Algorithm 2 Parallel algorithm for Blocked Bowtie Solver

Input: Matrix dimension N, Matrix A, Vector y, Block size C, Number of threads
Output: Vector x such that $Ax = y$
$n \leftarrow (N-1)/2$
$x[n] \leftarrow y[n]/A[n][0]$
times $\leftarrow n/C$ if $n\%C == 0$ else $n/C + 1$
omp_set_num_threads(num_threads)
for $i \in 0$ to *times* -1 **do**
    $bs \leftarrow i \times C + 1$
    $be \leftarrow \min(n, (i+1) \times C)$
    #pragma omp single
    for $k \in bs$ to be **do**
        $l \leftarrow n - k, r \leftarrow n + k$
        for $previous \in bs$ to $k - 1$ **do**
            Subtract contributions of already computed block variables from $y[l]$ *and* $y[r]$
        end for
        Solve the 2×2 *system using Cramer's rule to find* $x[l]$ *and* $x[r]$
    end for
    #pragma omp for schedule(dynamic, 32)
    for $row \in 0$ to $n - be - 1$ **do**
        for $current \in bs$ to be **do**
            Vectorized reduction subtraction of current block variables from upper rows of y
        end for
    end for
    #pragma omp for schedule(dynamic, 32)
    for $row \in n + be + 1$ to $N - 1$ **do**
        for $current \in bs$ to be **do**
            Vectorized reduction subtraction of current block variables from lower rows of y
        end for
    end for
end for

2.4 Performance Evaluation

2.4.1 Benchmark Method

We tested five input sizes, $n = 512, 1024, 2048, 4096, 8192$, with $p = 1, 2, 4, 8, 16$ threads. Each input-size and thread-count combination was run five times; the tables report the average execution time in microseconds. The matrix dimension is $N = 2n + 1$.

In the supplied solver, timing begins after input reading, memory allocation, and array initialization, and ends after the solution has been computed. Output writing and residual verification are outside the timed region. Speedup is measured relative to the same solver using one thread:

$$S_n(p) = \frac{\overline{T_n(1)}}{\overline{T_n(p)}}.$$

2.4.2 Results for n = 512, N = 1025

Threads	Average time (μs)	Speedup
1	374.600	1.000
2	402.800	0.930
4	411.600	0.910
8	439.600	0.852
16	504.600	0.742

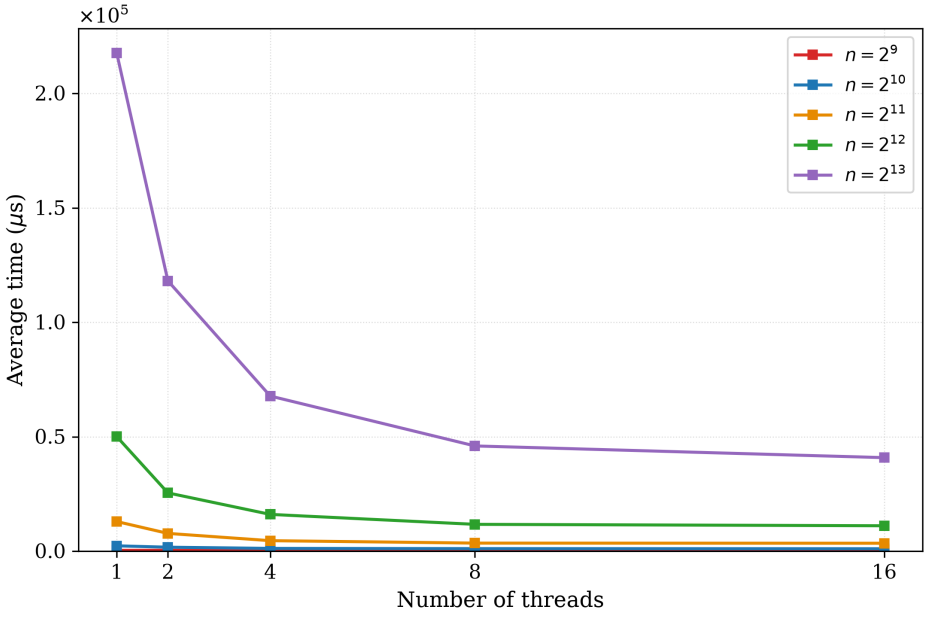


Figure 4: Average execution time versus number of threads.

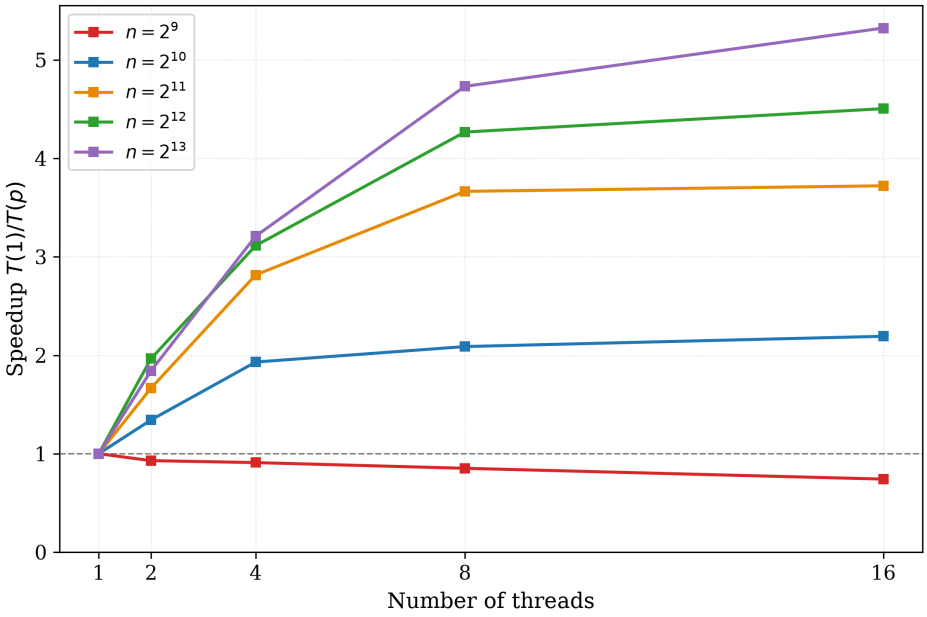


Figure 5: Speedup relative to execution with one thread.

2.4.3 Results for n = 1024, N = 2049

Threads	Average time (μs)	Speedup
1	2350.000	1.000
2	1749.600	1.343
4	1216.800	1.931
8	1125.200	2.089
16	1071.600	2.193

2.4.4 Results for n = 2048, N = 4097

Threads	Average time (μs)	Speedup
1	12959.000	1.000
2	7776.600	1.666
4	4602.000	2.816
8	3536.000	3.665
16	3482.200	3.721

2.4.5 Results for n = 4096, N = 8193

Threads	Average time (μs)	Speedup
1	50073.600	1.000
2	25471.600	1.966
4	16081.600	3.114
8	11734.600	4.267
16	11115.600	4.505

2.4.6 Results for n = 8192, N = 16385

Threads	Average time (μs)	Speedup
1	217602.200	1.000
2	118109.400	1.842
4	67791.600	3.210
8	45992.600	4.731
16	40873.200	5.324

2.4.7 Discussion of Observed Trends

- **Small input size:** For $n = 512$, time increases from $374.6\,\mu$s at one thread to $504.6\,\mu$s at sixteen threads. The resulting speedup is 0.742, indicating a slowdown. With this small workload, parallel-region, barrier, and scheduling overhead can outweigh the savings from dividing the row updates among threads. The timings are consistent with this explanation, although they do not isolate the individual overheads.
- **Benefit from larger workloads:** At sixteen threads, speedup increases to 2.193 for $n = 1024$, 3.721 for $n = 2048$, 4.505 for $n = 4096$, and 5.324 for $n = 8192$. Larger matrices provide more parallel update work relative to synchronization and scheduling costs.
- **Diminishing gains with more threads:** The reduction in execution time from eight to sixteen threads is approximately 4.8%, 1.5%, 5.3%, and 11.1% for $n = 1024$, 2048, 4096, and 8192, respectively. Doubling the thread count therefore does not halve runtime. The serial block solves, repeated barriers, and decreasing number of unsolved row pairs limit scaling. Memory throughput and the benchmark machine's core configuration may also contribute, but cannot be identified from the timing table alone.
- **Largest tested case:** For $n = 8192$, average runtime falls from approximately 217.6 ms to 40.9 ms. This gives the largest observed speedup, 5.324, with an efficiency of $S(16)/16 \approx 33.3\%$.

2.5 Compilation and Execution

Both programs use OpenMP and can be compiled using the following commands. The `-O3` flag enables compiler optimizations, `-fopenmp` enables OpenMP, and `-static-libstdc++` statically links the C++ standard library.

2.5.1 Question 1: Vertex and Edge Coloring

Compile:

```
g++ -O3 -fopenmp -static-libstdc++ q1_parallel.cpp -o q1
```

Run:

```
./q1 <input_file> <output_file> <num_threads>
```

2.5.2 Question 2: Hourglass Matrix Solver

Compile:

```
g++ -O3 -fopenmp -static-libstdc++ q2.cpp -o q2
```

Run:

```
./q2 <input_file> <output_file> <num_threads>
```

For both programs, the command-line